

2a)

Was sind die Gemeinsamkeiten und Unterschiede zwischen Kafka und MQTT-Brokern?

Beide Systeme implementieren ein Publisher/Subscriber Modell und ermöglichen asynchrone Kommunikation zwischen verteilten Systemen. Der große Unterschied ist in der Architektur von beiden. Während MQTT einen zentralen Broker nutzt, der Nachrichten filtert und weiterleitet, speichert Kafka hingegen Nachrichten dauerhaft in Topics und überlässt es dem Consumer, den Lesestand selbst zu verwalten. MQTT ist für ressourcenschwache Geräte und Kafka für Systeme mit hohem Volumen, die auch für dauerhafte Speicherung ausgelegt sind.

Für welche Szenarien ist Kafka besser geeignet?

Kafka eignet sich besser für Szenarien, wo man mit großen Datenmengen arbeitet, die dauerhaft gespeichert werden sollen und von mehreren Consumern unabhängig voneinander gelesen werden sollen, wie z.B. in Event-Sourcing, Echtzeit-Analysen oder Log-Aggregation

Man findet bei Vergleichen öfter die Aussage, Kafka würde das Modell „Dumb Broker / Smart Consumer“ implementieren, während bei MQTT „Smart broker / Dumb Consumer“ gilt. Was ist damit gemeint? Was muss ein Kafka-Consumer betrachten?

Bei Kafka ist der Broker passiv, also er speichert Nachrichten und kümmert sich nicht darum wer was gelesen hat. Der Consumer selbst muss seinen Offset verwalten, also merken wo er zuletzt aufgehört hat zu lesen. Bei MQTT ist es umgekehrt, also der Broker verwaltet aktiv die Subscriptions, filtert die Nachrichten und stellt sicher, dass Subscriber die richtigen Nachrichten erhalten. Der Consumer muss sich nicht darum kümmern.

Was sind Partitionen? Für was kann man sie neben Load-Balancing noch verwenden?

Partitionen sind Unterteilung eines Kafka-Topics. Nachrichten mit demselben Key landen immer in derselben Partition, wodurch die Reihenfolge garantiert wird. Neben Load-balancing können Partitionen auch für z.B. Parallelverarbeitung genutzt werden, also dass mehrere Consumer einer Consumer Group gleichzeitig verschiedene Partitionen lesen. Außerdem ermöglichen Partitionen eine gezielte Steuerung der Datenverteilung über mehrere Broker

2b) Weather Consumer

Es wurde ein Kafka Consumer implementiert, der das Topic *weather* abonniert und die Wetterdaten für Mosbach, Stuttgart und Bad Mergentheim ausgibt. Die Daten werden mittels der Jackson Library in ein *WeatherData*-Objekt geparkt und aus der Konsole ausgegeben.

Probleme:

- Feldname falsch, also zuerst wurden die Felder in der Klasse *WeatherData* anders benannt als im JSON, deswegen konnte Jackson die Werte nicht zuordnen und hat alle Felder falsch ausgegeben. Als Lösung wurden die Rohdaten erst mit `System.out.println(record.value())` ausgegeben um die echten Feldnamen zu ermitteln.

2c)

Wozu dienen Key und Value beim Versenden von Kafka-Messages?

Der Key bestimmt in welche Partitionen eine Nachricht geschrieben wird. Nachrichten mit demselben Key landen immer in derselben Partition, was die Reihenfolge garantiert. Der Value enthält den eigentlichen Nachrichteninhalt, in diesem Fall wäre es ein JSON-String mit den jeweiligen Nutzdaten.

Es wurde ein Kafka-Producer implementiert der alle 5 Sekunden Systemdaten (CPU-Last und Arbeitsspeicher) an das Topic *vlvs_inf23_RKC* sendet. Dabei wird ein JSON-String als Value und „System“ als Key verwendet

Probleme:

- CPU-Last unter Windows: Die Methode `getSystemLoadAverage()` gibt unter Windows immer -1.0 zurück, da sie auf dem Betriebssystem nicht unterstützt wird.

2d)

Ein Kafka-Consumer wurde implementiert, der die Wetterdaten aus dem Topic weather liest und an die Graphite-Instanz des DHBW-Servers weiterleitet. Anschließend wurden die Daten in einem Grafana Dashboard visualisiert. Screenshot: AuchProject\Aufgabe2\Grafana_RKC.png